```
from collections import deque
import copy

goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

moves = [(0,1),(1,0),(0,-1),(-1,0)]
move_names = ["Right","Down","Left","Up"]

def find_blank(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                return i,j

def print_board(board):
    for row in board:
        print(row)
    print()

def bfs(initial):

    queue = deque()
    visited = set()

    queue.append((initial,[]))
    visited.add(str(initial))

    while queue:

        state,path = queue.popleft()

        if state == goal:
            return path + [state]

        x,y = find_blank(state)

        for move in moves:
            new_x = x + move[0]
            new_y = y + move[1]

            if 0<=new_x<3 and 0<=new_y<3:

                new_state = copy.deepcopy(state)

                new_state[x][y],new_state[new_x][new_y] = new_state[new_x][new_y],new_state[x][y]

                if str(new_state) not in visited:
                    visited.add(str(new_state))
                    queue.append((new_state,path+[state]))

    return None


initial = [[1,2,3],
           [4,0,6],
```

```
        [7,5,8]]

solution = bfs(initial)

if solution:
    print("Solution Path:\n")
    for step in solution:
        print_board(step)
else:
    print("No solution found")
```

Output:

```
Solution Path:

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]


=== Code Execution Successful ===
```